

```
1: /**
2:  * @file quadtree.c
3:  * @brief Implementaci3n recursiva para contar pÃ-xeles negros en un Quadtree
4:  * @author Asistente IA
5:  * @date 2026
6:  *
7:  * Este programa lee una cadena que representa un quadtree y calcula
8:  * recursivamente el nÃºmero total de pÃ-xeles negros en una imagen de 32x32.
9:  */
10:
11: #include <stdio.h>
12: #include <stdlib.h>
13: #include <string.h>
14:
15: /**
16:  * @brief Calcula recursivamente los pÃ-xeles negros en un quadtree
17:  *
18:  * La funci3n recorre la cadena del quadtree usando un Ã-ndice pasado por referen
19:  cia.
20:  * En cada nodo:
21:  * - Si es 'f': retorna el Ã-rea total del cuadrante actual
22:  * - Si es 'e': retorna 0
23:  * - Si es 'p': suma recursivamente los resultados de sus 4 hijos
24:  *
25:  * @param tree Cadena que representa el quadtree (ej: "ppeeefpffeefe")
26:  * @param index Ã-ndice actual en la cadena (se pasa por referencia para avanza
27:  r)
28:  *
29:  * @param size TamaÃ±o del cuadrante actual en pÃ-xeles (ancho o alto)
30:  * @return int NÃºmero de pÃ-xeles negros en el cuadrante actual
31:  *
32:  * @note El parÃ-metro size se reduce a la mitad en cada nivel recursivo
33:  * @note Cuando size=1, representa un solo pÃ-xel
34:  */
35: int countBlackPixels(const char *tree, int *index, int size) {
36:     // --- CASOS BASE ---
37:
38:     // Caso base 1: Nodo 'f' (full) - todos los pÃ-xeles son negros
39:     if (tree[*index] == 'f') {
40:         // Avanzamos al siguiente carÃ-cter para la prÃ³xima llamada
41:         (*index)++;
42:         // El Ã-rea total es size * size (pÃ-xeles negros en este cuadrante)
43:         return size * size;
44:     }
45:
46:     // Caso base 2: Nodo 'e' (empty) - todos los pÃ-xeles son blancos
47:     if (tree[*index] == 'e') {
48:         // Avanzamos al siguiente carÃ-cter
49:         (*index)++;
50:         // No hay pÃ-xeles negros en este cuadrante
51:         return 0;
52:     }
53:
54:     // --- CASO RECURSIVO: Nodo 'p' (parent) - tiene 4 hijos ---
55:     if (tree[*index] == 'p') {
56:         // Avanzamos al primer hijo
57:         (*index)++;
58:
59:         // El nuevo tamaÃ±o para cada hijo es la mitad (porque dividimos el cuadra
60:         nte)
61:         int childSize = size / 2;
62:
63:         // Variable para acumular el total de pÃ-xeles negros de los 4 hijos
64:         int totalBlackPixels = 0;
65:
66:         // --- Procesamos los 4 hijos recursivamente ---
67:         // Orden de los hijos segÃºn la figura: NO, NE, SO, SE
68:         // (Noroeste, Noreste, Suroeste, Sureste)
69:
70:         // Hijo 1: Cuadrante Noroeste (top-left)
71:         totalBlackPixels += countBlackPixels(tree, index, childSize);
```

```
68:
69:     // Hijo 2: Cuadrante Noreste (top-right)
70:     totalBlackPixels += countBlackPixels(tree, index, childSize);
71:
72:     // Hijo 3: Cuadrante Suroeste (bottom-left)
73:     totalBlackPixels += countBlackPixels(tree, index, childSize);
74:
75:     // Hijo 4: Cuadrante Sureste (bottom-right)
76:     totalBlackPixels += countBlackPixels(tree, index, childSize);
77:
78:     // Retornamos la suma de los p  xeles negros de los 4 cuadrantes
79:     return totalBlackPixels;
80: }
81:
82:     // Si llegamos aqu  , hay un car  cter inv  lido en la entrada
83:     // (esto no deber  a ocurrir si la entrada es v  lida)
84:     fprintf(stderr, "Error: Car  cter inv  lido '%c' en el quadtree\n", tree[*inde
x]);
85:     return -1;
86: }
87:
88: /**
89:  * @brief Funci  n wrapper que inicia el conteo de p  xeles negros
90:  *
91:  * @param tree Cadena que representa el quadtree
92:  * @return int N  mero total de p  xeles negros en la imagen de 32x32
93:  */
94: int getBlackPixels(const char *tree) {
95:     int index = 0; //   ndice inicial en la cadena
96:
97:     // La imagen es de 32x32 p  xeles
98:     // El   rea total inicial es 32 * 32 = 1024 p  xeles
99:     int imageSize = 32;
100:
101:     return countBlackPixels(tree, &index, imageSize);
102: }
103:
104: /**
105:  * @brief Versi  n alternativa con explicaci  n paso a paso (para depuraci  n)
106:  *
107:  * @param tree Cadena del quadtree
108:  * @param index   ndice actual (por referencia)
109:  * @param size Tama  o del cuadrante
110:  * @param depth Profundidad actual (para indentaci  n en depuraci  n)
111:  * @return int N  mero de p  xeles negros
112:  */
113: int countBlackPixelsDebug(const char *tree, int *index, int size, int depth) {
114:     // Imprimir indentaci  n seg  n la profundidad
115:     for (int i = 0; i < depth; i++) {
116:         printf(" ");
117:     }
118:
119:     char nodeType = tree[*index];
120:     printf("Nodo: %c, Tama  o: %dx%d,   rea: %d\n",
121:           nodeType, size, size, size * size);
122:
123:     // Caso base: f
124:     if (nodeType == 'f') {
125:         (*index)++;
126:         printf("%s   Cuadrante completamente negro: +%d p  xeles\n",
127:               depth * 2, "", size * size);
128:         return size * size;
129:     }
130:
131:     // Caso base: e
132:     if (nodeType == 'e') {
133:         (*index)++;
134:         printf("%s   Cuadrante completamente blanco: +0 p  xeles\n",
135:               depth * 2, "");
136:         return 0;
```

```
137:     }
138:
139:     // Caso recursivo: p
140:     if (nodeType == 'p') {
141:         (*index)++;
142:         int childSize = size / 2;
143:         printf("%sâ\206\222 Cuadrante mixto, dividiendo en 4 subcuadrantes de %dx
%d\n",
144:             depth * 2, "", childSize, childSize);
145:
146:         int total = 0;
147:
148:         // Procesar hijos (Noroeste, Noreste, Suroeste, Sureste)
149:         printf("%s Procesando Noroeste:\n", depth * 2, "");
150:         total += countBlackPixelsDebug(tree, index, childSize, depth + 1);
151:
152:         printf("%s Procesando Noreste:\n", depth * 2, "");
153:         total += countBlackPixelsDebug(tree, index, childSize, depth + 1);
154:
155:         printf("%s Procesando Suroeste:\n", depth * 2, "");
156:         total += countBlackPixelsDebug(tree, index, childSize, depth + 1);
157:
158:         printf("%s Procesando Sureste:\n", depth * 2, "");
159:         total += countBlackPixelsDebug(tree, index, childSize, depth + 1);
160:
161:         printf("%sâ\206\222 Total del cuadrante: %d pÃ-xeles negros\n",
162:             depth * 2, "", total);
163:
164:         return total;
165:     }
166:
167:     return -1;
168: }
169:
170: /**
171:  * @brief FunciÃ³n principal del programa
172:  *
173:  * Lee una cadena del usuario, cuenta los pÃ-xeles negros usando recursiÃ³n
174:  * y muestra el resultado.
175:  *
176:  * @return int 0 si el programa termina correctamente
177:  */
178: int main() {
179:     char tree[1000]; // Buffer para almacenar la cadena del quadtree
180:
181:     printf("=== CONTADOR DE PÃ\215XELES NEGROS EN QUADTREE ===\n");
182:     printf("El quadtree representa una imagen de 32x32 pÃ-xeles (1024 total)\n");
183:     printf("SÃmbolos:\n");
184:     printf(" 'p' = parent (nodo padre, tiene 4 hijos)\n");
185:     printf(" 'f' = full (cuadrante completamente negro)\n");
186:     printf(" 'e' = empty (cuadrante completamente blanco)\n");
187:     printf("\nEjemplo de entrada: ppeeefffeefe\n");
188:     printf("Ejemplo de salida esperada: 448\n\n");
189:
190:     while (1) {
191:         printf("Ingrese la representaciÃ³n del quadtree (o 'salir' para terminar):
");
192:
193:         // Leer la lÃnea completa
194:         fgets(tree, sizeof(tree), stdin);
195:
196:         // Eliminar el carÃcter de nueva lÃnea si estÃ presente
197:         size_t len = strlen(tree);
198:         if (len > 0 && tree[len - 1] == '\n') {
199:             tree[len - 1] = '\0';
200:         }
201:
202:         // Verificar si el usuario quiere salir
203:         if (strcmp(tree, "salir") == 0) {
204:             printf("Ã¡Hasta luego!\n");
```

```
205:         break;
206:     }
207:
208:     // Verificar que la cadena no esté vacía
209:     if (strlen(tree) == 0) {
210:         printf("Por favor ingrese una cadena válida.\n\n");
211:         continue;
212:     }
213:
214:     // --- Versión normal (sin debug) ---
215:     int result = getBlackPixels(tree);
216:     printf("\nResultado (versión normal):\n");
217:     printf("Número de píxeles negros: %d\n", result);
218:
219:     // --- Versión con depuración (opcional) ---
220:     printf("\n--- Proceso detallado (debug) ---\n");
221:     int index = 0;
222:     int debugResult = countBlackPixelsDebug(tree, &index, 32, 0);
223:     printf("--- Fin del proceso detallado ---\n");
224:     printf("Resultado (debug): %d\n\n", debugResult);
225: }
226:
227: return 0;
228: }
```